



Core Programming & Core OOPS Foundations Quick Review, Arrays & Strings			
Day 1 - Programming & Data Structures			
Difficulty	Problem	Topic	Pattern
Medium	Employee Management System	Core OOPS	Classes, Encapsulation, Inheritance, Polymorphism
Easy	Missing Number in Attendance Roll	Arrays	Mathematical Array Pattern
Medium	Customer Login Session Analyzer	Strings	Character Index Mapping / Last Seen Position
Easy	Reverse Transaction Log String	Strings	Two Pointers
Medium	Maximum Subarray Profit	Arrays	Kadane's Algorithm
Hard	Merge Overlapping Meeting Slots	Arrays/Intervals	Sort + Merge Intervals
Hard	Product Array Except Self	Arrays	Prefix Sum / Suffix Product

Stack, Queue and Priority Queue & Heap			
Day 2 - Data Structures & Recursion			
Difficulty	Problem	Topic	Pattern
Easy	Validate Bracket Sequence	Stack	Stack Validation
Medium	Customer Support Ticket Scheduler	Queue/Deque	Priority Queue + Heap
Medium	Browser Navigation History Manager	Queue + DLL	Stack Design
Medium	Next Greater Price Alert	Stack	Monotonic Stack (Next Greater Element)
Hard	Top Trending Hashtags Tracker	Heap	Top K Elements (Min Heap)
Hard	Live Stock Price Monitor	Heap	Kth Largest in Stream
Hard	Sliding Window Maximum	Monotonic Deque	Advanced Queue
Hard	Generate Heap Problem	Heap	Priority Queue

**Linked List, Recursion, Hashing/Dictionary & Matrix Problems****Day 3 - Data Structures & Matrix Problems**

Difficulty	Problem	Topic	Pattern
Easy	First Non-Repeating Character	Hashing	Frequency Counting
Medium	Reorder Customer Delivery Route	Linked List	Reverse + Middle + Merge
Medium	Product Locator in Smart Warehouse	Matrix	Search in Sorted Matrix
Medium	Generate All Subsets	Recursion	Backtracking Foundation
Hard	Word Search	Recursion + Backtracking	DFS Backtracking
Hard	Warehouse Inventory Spiral Scan	Matrix	Spiral Matrix Traversal
Hard	Fraudulent Transaction Detector	Hashing	HashSet Lookup
Hard	Customer Order Cache Synchronization	Linked List + HashMap	Copy Linked List with Random Pointer

Sliding Window, Two Pointers & Sorting**Day 4 - Sliding Window, Two Pointers & Sorting**

Difficulty	Problem	Topic	Pattern
Easy	Sort Employee Salaries	Sorting	Insertion Sort
Medium	Network Traffic Analyzer	Sliding Window	Longest Substring Without Repeating Characters
Medium	Water Container Optimization	Two Pointer Style Scan	Container With Most Water
Hard	Distinct Elements in Every Window	Sliding Window	Fixed-Size Window + HashSet
Hard	Online Exam Score Ranking	Sorting	Quick Sort Partition
Hard	Count Inversions	Sorting	Merge Sort Application
Hard	Merge Overlapping Intervals	Sorting + Intervals	Sort & Merge



Searching, Binary Search, Advanced OOPS			
Day 5 - Searching, Binary Search, Advanced OOPS			
Difficulty	Problem	Topic	Pattern
Easy	Binary Search Variant (New)	Searching	Classic Binary Search
Medium	Product Inventory Range Query	Binary Search	Lower Bound / Upper Bound
Medium	Cloud Server Capacity Planner	Binary Search on Answer	Search in Rotated Sorted Array
Medium	Book Allocation Problem	Binary Search on Answer	Min-Max Binary Search on Answer
Hard	Online Food Delivery Pricing Engine	Advanced OOPS	Factory Design Pattern
Hard	Payment Gateway Design	Advanced OOPS Usage	Abstraction, Interface, Strategy Pattern
Hard	Ride Booking System Architecture	Advanced OOPS	Inheritance + Interface + Strategy Pattern

Trees & Greedy Algorithms			
Day 6 - Trees & Greedy Algorithms			
Difficulty	Problem	Topic	Pattern
Easy	Count Nodes in Tree	Trees	DFS Recursion
Medium	Railway Platform Scheduler	BST & Trees	Minimum Platforms
Medium	Validate BST	BST	Range Validation
Medium	Organization Hierarchy Common Manager	Trees	LCA in Binary Tree
Hard	Cloud Resource Allocation Planner	Greedy	Fractional Knapsack
Hard	Smart City Surveillance View	Trees	Right Side View of Binary Tree
Hard	Serialize/Deserialize Tree	Trees	Design + Traversal
Hard	Huffman Encoding for File Compression	Greedy	Huffman Coding (Min Heap + Greedy)



Graphs & Dynamic Programming			
Day 7 — Graphs & Dynamic Programming			
Difficulty	Problem	Topic	Pattern
Easy	Count Connected Components	Graph	DFS/BFS
Medium	University Course Planner	Graph	Topological Sort
Medium	Social Network Friend Recommendation	Graph	BFS Traversal
Medium	GPS Route Optimizer	Graph	Dijkstra Shortest Path
Medium	Longest Common Subsequence	DP	2D DP
Hard	Minimum Spanning Tree	Graph	Union Find
Hard	Network Delay Time	Graph	Bellman-Ford / SSSP
Hard	Digital Wallet Coin Optimizer	DP	Coin Change DP
Hard	Stock Growth Analyzer	DP	Longest Increasing Subsequence
Hard	Minimum Path Cost in Grid	DP	2D Grid DP (Min Cost Path)
Hard	0/1 Knapsack	DP	Classic DP



Problem Statements – Day by Day:

Day 1:

1. A multinational company wants to build an internal employee management portal to manage developers, testers, and managers efficiently. Each employee type has common properties like employee ID and salary, while specific roles have additional responsibilities and behaviors. The HR team wants the system to demonstrate proper object-oriented principles for scalability and maintainability. The application should support inheritance, polymorphism, and encapsulation for future expansion.
2. A school administration system stores attendance records for students with roll numbers from 1 to N. During attendance submission, one roll number was accidentally skipped due to a network issue. The administration wants to quickly identify the missing student record without sorting the data. The solution should work efficiently even for very large class sizes.
3. An e-commerce company tracks customer login sessions to analyze user engagement patterns. The analytics team wants to determine the longest browsing session where no login token or character repeats. This helps in identifying uninterrupted user activity and improving personalized recommendations. The solution should handle millions of session logs efficiently.
4. A banking application stores transaction history as encrypted log strings for audit purposes. During debugging and fraud investigation, administrators need to reverse the transaction logs efficiently. The system should process long transaction strings quickly without excessive memory usage. The functionality must support real-time log analysis.
5. A stock trading platform records daily profit and loss values for different trading sessions. Financial analysts want to identify the continuous period that generated the maximum profit. This helps investors understand the best-performing trading window. The system should efficiently process very large datasets containing both positive and negative values.
6. A corporate office booking system manages meeting room schedules throughout the day. Multiple employees may accidentally book overlapping meeting slots for the same conference room. The scheduling system must merge overlapping intervals to optimize room utilization and avoid conflicts. The solution should efficiently support thousands of meetings daily.
7. A large-scale e-commerce platform maintains real-time inventory valuation data across thousands of warehouses. For financial reporting, the operations team needs to compute, for every warehouse, the combined valuation of all other warehouses excluding the current one, to detect anomalies in regional stock distribution. Since some warehouses may report a valuation of zero due to stockouts, the computation cannot use division. Given the scale of the operation (up to 10^5 warehouses with values updated frequently), the solution must run in linear time using prefix and suffix product calculations, without using nested loops or recomputing totals from scratch for each warehouse.



Day 2:

1. A compiler development team is building a syntax validation module for a programming language editor. The module must verify whether brackets in user-written code are balanced correctly. Invalid expressions should be detected immediately to provide meaningful error messages. The system should efficiently process large source code files.
2. A customer support platform receives thousands of tickets every hour from users worldwide. Tickets with higher priority and earlier timestamps must be resolved first to maintain SLA commitments. The company wants an automated scheduling engine that processes tickets efficiently. The scheduler should dynamically manage ticket priorities in real time.
3. A web browser company wants to implement navigation history functionality similar to modern browsers. Users should be able to move backward and forward between previously visited pages seamlessly. The system must maintain browsing history efficiently even after thousands of navigation operations. Performance and memory optimization are critical requirements.
4. A stock trading platform monitors daily closing prices for a stock and wants to generate an alert for each trading day indicating the next future day on which the price exceeds that day's closing price. This helps traders identify upcoming price recovery points without manually scanning the entire price history. If no such future day exists for a given day, the system should mark it accordingly (e.g., with -1). Since the platform processes price histories with up to 10^5 entries and needs near-instant alert generation, a brute-force comparison of every pair of days would be too slow — the solution must compute all "next greater price" alerts in a single linear pass using an efficient stack-based approach.
5. A social media company monitors millions of hashtags posted by users every minute. The analytics engine must continuously identify the top trending hashtags in real time. This information is used to power recommendation feeds and trending sections. The solution should efficiently handle high-frequency streaming data.
6. A financial analytics platform receives continuous stock price updates from the stock exchange. Investors want to monitor the K-th largest stock price dynamically as new prices arrive. The system should provide near real-time updates with minimal latency. Efficient handling of streaming data is essential for accurate financial monitoring.
7. A cloud infrastructure monitoring system tracks CPU utilization every second across servers. System administrators need to determine the maximum utilization within every fixed monitoring window. This helps detect sudden spikes and abnormal server load patterns. The application should efficiently process large real-time streams.
8. A task scheduling service prioritizes jobs based on urgency and execution deadlines. The company wants to organize tasks internally using a heap data structure for efficient retrieval. Higher-priority tasks should always be processed before lower-priority ones. The scheduler must support fast insertion and extraction operations.



Day 3:

1. A live chat application processes continuous streams of customer messages. The moderation system wants to identify the first non-repeating character in each message for pattern analysis. This functionality is also useful for spam detection and text analytics. The system should efficiently process large volumes of text data.
2. A logistics company stores delivery locations as a linked route sequence. To optimize delivery efficiency, the company wants to reorder routes in a specific pattern before dispatching delivery agents. The system should perform the reordering in-place without additional memory usage. Efficient handling of long delivery chains is required.
3. A smart warehouse stores products in a sorted grid-based inventory system. Warehouse robots must quickly locate products based on product IDs for faster order fulfillment. The search operation should avoid scanning the entire warehouse manually. The system should support efficient product lookup even in very large matrices.
4. An online retail platform wants to generate all possible product bundle combinations for marketing campaigns. Each bundle can contain different combinations of available products. Marketing teams use these combinations to create discount offers and recommendations. The system should efficiently generate all possible subsets for small-to-medium product lists.
5. A document analysis platform scans large character grids to detect hidden keywords and patterns. Security analysts use the tool to identify suspicious or restricted words in scanned documents. The search engine must support horizontal and vertical traversal with backtracking capability. The solution should efficiently search across multiple document grids.
6. An automated warehouse robot scans storage racks arranged in matrix form. The robot follows a spiral traversal pattern to inspect all inventory locations systematically. This approach minimizes repeated movement and improves scanning efficiency. The system should support large warehouse layouts with optimized traversal logic.
7. A banking system processes millions of online transactions every day. The fraud detection engine must identify duplicate or suspicious transaction IDs instantly to prevent financial fraud. Security teams rely on fast lookup mechanisms to minimize monetary risk and unauthorized access. The solution should efficiently detect duplicates in large-scale transaction streams.
8. An e-commerce platform stores customer order history in a linked structure with additional random references for recommendation mapping. During server migration, the company wants to create an exact deep copy of the order cache without affecting original references. The synchronization process must preserve all relationships accurately. The system should efficiently handle large linked structures with complex dependencies.



Day 4:

1. An HR analytics platform processes employee salary records for payroll generation and performance reviews. The system must sort salary data efficiently to generate reports such as highest-paid employees and department-wise rankings. Since payroll data can grow significantly in large organizations, optimized sorting performance is essential. The application should support real-time reporting requirements.
2. A cybersecurity monitoring platform analyzes incoming network requests from multiple servers. The system wants to identify the longest continuous traffic pattern without repeated request signatures. This helps security engineers detect stable traffic sessions and unusual network behavior. The analysis engine should efficiently process high-frequency streaming traffic logs.
3. An infrastructure company is designing water storage units using vertical walls of varying heights. Engineers want to determine the maximum amount of water that can be stored between any two walls. The optimization process should minimize unnecessary calculations for large datasets. Efficient computation is critical for large-scale construction planning.
4. A web analytics platform processes a continuous stream of user IDs representing page views on a website. For traffic quality monitoring, the platform needs to determine the number of distinct users within every window of K consecutive page views as the stream progresses, in order to flag periods of suspiciously low diversity (possible bot traffic) or unusually high engagement variety. Given the high volume of page-view data (up to 10^5 entries) arriving continuously, recomputing the distinct count from scratch for every window would be far too slow. The solution must maintain a running count of distinct users as the fixed-size window slides one position at a time, using a frequency map / hashset to efficiently add and remove elements at each step.
5. An online learning platform conducts coding assessments for thousands of students simultaneously. The platform must rank students based on their scores quickly and accurately after exam completion. Efficient sorting and ranking are essential to display leaderboards in real time. The system should scale for large batches of exam submissions.
6. A leaderboard analytics system wants to measure how far a ranking list is from being completely sorted. Every inversion represents a pair of users whose rankings are incorrectly ordered. The company uses this metric to analyze ranking instability and recommendation quality. The system should efficiently process large ranking datasets.
7. A hotel booking platform receives reservation intervals from customers throughout the day. Some bookings overlap due to duplicate or adjacent reservation requests. The platform wants to merge these intervals to optimize room allocation and avoid conflicts. The solution should efficiently manage thousands of booking records.



Day 5:

1. A digital library stores millions of books sorted by unique identifiers. Users frequently search for books using these IDs, and the platform needs a highly efficient lookup mechanism. The search engine should minimize lookup time while maintaining accuracy. The system must support extremely large sorted datasets.
2. A warehouse management system maintains sorted inventory quantities for products across multiple storage centers. Business analysts want to find the starting and ending positions of a specific inventory quantity efficiently. This helps generate stock reports and demand analysis quickly. The platform should handle large-scale inventory databases efficiently.
3. A cloud computing company allocates servers dynamically based on incoming workloads. Engineers want to determine the minimum server capacity required to process all tasks within a given deadline. The optimization process should reduce operational costs while maintaining performance guarantees. The system should efficiently handle very large workload datasets.
4. An online publishing platform needs to distribute N manuscripts, each with a known number of pages, among M editors for proofreading before a deadline. Each editor must be assigned a contiguous block of manuscripts (no manuscript can be split between editors), and every editor must receive at least one manuscript. To keep the review process fair and avoid bottlenecks, the platform wants to minimize the maximum total page count assigned to any single editor. Since the number of manuscripts can be large (up to 10^5) with widely varying page counts, checking every possible way to split the manuscripts would be far too slow. The solution must efficiently find the minimum possible value for the "maximum pages per editor" by searching over possible answer values and verifying feasibility at each step.
5. A food delivery platform supports multiple pricing models such as festive discounts, premium delivery charges, and membership offers. The engineering team wants to design a flexible pricing engine using object-oriented design patterns. New pricing strategies should be added without modifying existing business logic. The system must support scalability and clean architecture principles.
6. A fintech company supports multiple payment methods including UPI, cards, wallets, and net banking. The platform wants to design a payment gateway that allows seamless integration of new payment providers. Engineers aim to use abstraction and strategy design patterns for maintainability and scalability. The system should provide secure and extensible payment processing.
7. A ride-sharing company manages ride requests, driver assignments, and fare calculations across multiple cities. The platform wants to design a scalable ride booking architecture using object-oriented principles. Different ride types such as economy, premium, and bike rides should support customized pricing logic. The system should remain extensible for future transportation services.



Day 6:

1. A cloud file storage platform organizes folders and files in a hierarchical tree structure. System administrators want to calculate the total number of nodes present in the structure efficiently. This helps estimate storage complexity and resource utilization. The solution should support very deep hierarchical trees.
2. A railway station handles arrival and departure schedules for multiple trains every day. Station managers want to determine the minimum number of platforms required to avoid train delays and collisions. Efficient scheduling helps improve station management and passenger experience. The system should process large train schedules efficiently.
3. A database indexing system internally stores records using binary search tree structures. Engineers need to verify whether the generated tree satisfies valid BST properties before executing search operations. Incorrect structures may significantly impact search performance and data consistency. The validation process should efficiently handle large trees.
4. A multinational company maintains its employee hierarchy using a tree-like organizational structure. HR managers frequently need to determine the lowest common manager between two employees for approval workflows. The system should quickly identify shared reporting authorities in the hierarchy. Efficient traversal is essential for very large organizations.
5. A cloud services company wants to maximize profit while allocating limited computing resources to clients. Each task requires a certain number of resources and generates specific revenue. The allocation system should choose the most profitable combination within capacity limits. Efficient optimization is necessary for large enterprise workloads.
6. A smart city surveillance department monitors buildings using cameras placed strategically across the city. Engineers want to determine which buildings are visible from the right-side perspective for monitoring purposes. This visibility analysis helps optimize surveillance coverage and camera placement. The system should efficiently process large hierarchical layouts.
7. A distributed storage platform transfers hierarchical data structures between servers over the network. Engineers need a reliable mechanism to convert tree structures into transferable formats and reconstruct them later. Data integrity and reconstruction accuracy are critical requirements. The serialization process should support efficient storage and transmission.
8. A cloud storage company is building a file compression utility to reduce storage costs for large text-based log files. Given the frequency of occurrence of each character in a file, the system needs to assign each character a variable-length binary code such that no code is a prefix of another character's code (so the compressed data can be decoded unambiguously), while minimizing the overall size of the compressed output. Frequently occurring characters should get shorter codes and rare ones longer codes. Since log files can contain a wide variety of characters with highly skewed frequency distributions and may be very large, the encoding scheme must be built efficiently by repeatedly merging the two least frequent symbols using a min-heap, continuing until a single optimal encoding tree is formed.



Day 7:

1. A network infrastructure company manages multiple interconnected systems across data centers. Engineers want to determine how many isolated subnetworks exist within the infrastructure. This analysis helps identify disconnected systems and optimize connectivity planning. The solution should efficiently process large graph-based networks.
2. A university maintains prerequisite relationships between courses offered across departments. Students want to know the valid order in which courses can be completed without violating prerequisites. The academic planning system should detect dependency conflicts automatically. Efficient graph traversal is essential for large course catalogs.
3. A social media platform wants to recommend potential friends based on mutual connections and graph traversal. The recommendation engine analyzes user relationships to improve engagement and networking opportunities. Suggestions should prioritize users connected through shorter relationship paths. The system must efficiently process millions of users and connections.
4. A navigation application calculates the shortest route between locations using road network data. Drivers rely on the application for accurate travel distance and optimized navigation guidance. The routing engine should efficiently process weighted road graphs with thousands of intersections. Real-time route optimization is essential for user experience.
5. A plagiarism detection platform compares documents submitted by users across multiple sources. The system wants to identify the longest matching subsequence between two text files for similarity analysis. This helps detect copied content while preserving sequence order information. Efficient comparison is necessary for large document repositories.
6. A telecom company plans to connect multiple cities using network cables at minimum installation cost. Engineers want to ensure all cities remain connected while minimizing total infrastructure expenses. The solution should identify the optimal set of connections without creating cycles. Efficient graph processing is critical for large-scale networks.
7. A telecommunications company operates a network of N servers connected by direct links, each with a known signal transmission delay. When a system alert is broadcast from a designated source server, the operations team needs to determine the minimum time required for the signal to reach every other server in the network, assuming it travels along the fastest combination of links. If even one server cannot be reached from the source, the broadcast is considered incomplete and the system should report this clearly. Given that the network can contain a large number of servers and links with varying delays, the solution must compute the shortest propagation time from the source to all other servers efficiently, then determine the overall time for the entire network to receive the alert (or detect that some servers are unreachable).



8. A digital wallet platform offers cashback rewards using predefined coin denominations. The system must determine the minimum number of coins required to generate a target cashback amount. Efficient optimization helps reduce operational overhead and improve transaction speed. The platform should support large cashback values efficiently.
9. An investment analytics company studies stock performance over long trading periods. Financial analysts want to identify the longest increasing growth trend in stock prices. This helps investors understand stable growth opportunities and market confidence. The solution should efficiently process large financial datasets.
10. A logistics company models its delivery region as a grid, where each cell represents a zone with an associated cost (fuel, tolls, or time penalties) for a delivery vehicle to pass through. A vehicle starts at the top-left zone and must reach the bottom-right zone, moving only right or downward at each step (no backtracking). The company wants to determine the minimum total cost route from start to destination across the entire grid. Since the delivery region can be represented as a large grid (up to 1000 x 1000 zones), evaluating every possible path individually would be computationally infeasible. The solution must build up the minimum cost to reach each cell from the costs of the cells it can be reached from, ultimately determining the minimum cost to reach the destination in a single efficient pass.
11. A cargo logistics company transports packages with limited vehicle carrying capacity. Each package has a weight and associated profit value, and the company wants to maximize overall revenue. Since items cannot be divided, each package can either be selected or skipped completely. The optimization engine should efficiently determine the best package combination.